



Lab7-模糊测试技术实验

姓名：李汶峰

班级：23 级网安 1 班

学号：202300460158

指导教师：刁文瑞

2025 年 12 月 28 日

目录

1 诚信声明	2
2 Task 1: 初探 AFL	2
2.1 环境准备	2
2.2 初步测试	2
3 Task 2: 基于 AFL++ 测试 Xpdf	4
3.1 漏洞复现	4
3.2 漏洞分析	7
4 Task 3: 使用 QEMU 模式执行模糊测试	7

1 诚信声明

我承诺，本实验报告中的所有内容均为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 Task 1: 初探 AFL

2.1 环境准备

首先进行源码形式构建，克隆 AFL 最新版本的源码，随后进入 AFL 文件夹进行安装：

```
1 git clone https://github.com/google/AFL.git
2 make
3 sudo make install
```

2.2 初步测试

首先准备一段待测试的代码 test.c，此代码中模拟了 4 个漏洞，需要使用 AFL 工具进行模糊测试。使用 afl-gcc 编译需要测试的 test.c 文件：

```
1 afl-gcc-g-o test test.c
```

然后创建两个目录 fuzz_in 和 fuzz_out，分别用于存放模糊测试的输入和输出。在 fuzz_in 目录中准备一个测试用例，最后执行命令进行模糊测试：

```
1 echo 'aaa' > ./fuzz_in/case
2 afl-fuzz-i ./fuzz_in-o ./fuzz_out ./test
```

模糊测试的结果如图所示，总共发现了 7 个 uniq crashes：

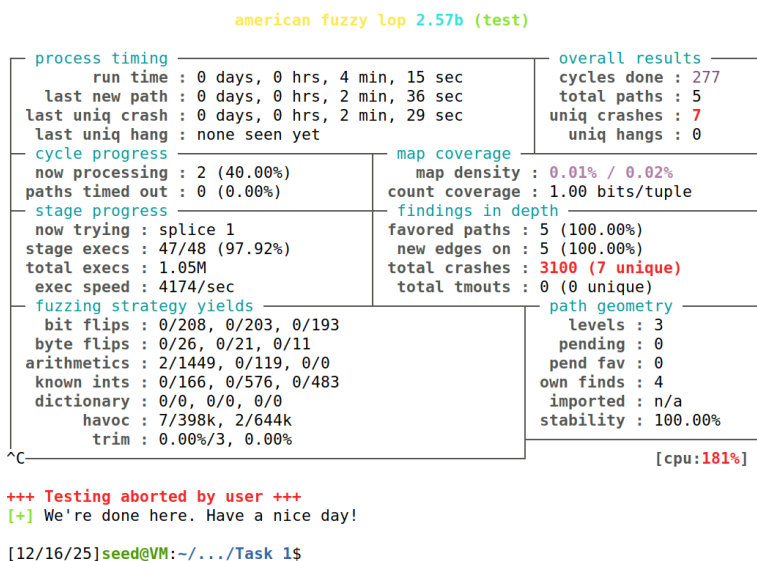


图 1: TASK1 模糊测试运行图

fuzz_out 目录记录了模糊测试的运行结果。通过分析此目录里导致程序崩溃的具体测试用例，使用 xxd 查看之前触发 crash 的 7 个例子，分析如下。

首先 id 为 2 的样本发生的是 SIGSEGV 段错误，即代码中主动调用的 raise(SIGSEGV)，当数组第一个字节为 C 且总长度为 30 字节时主动触发段错误。通过 xxd 分析样本发现，其首字节为十六进制 43，对应的 ASCII 码就是 C，同时总字节数为 30，所以对应的就是代码中主动触发的段错误。

```
1 if(str[0] == 'C' && len == 30) {  
2     raise(SIGSEGV);  
3 }
```

```
[12/16/25]seed@VM:~/.../crashes$ xxd id:000002,sig:11,src:000001,op:havoc,rep:4  
00000000: 4343 4343 4333 4343 4343 4343 435a 4343  CCCCCCCCCCCCZCC  
00000010: 4343 4343 4343 4343 4343 4343 6161      CCCCCCCCCCaa  
[12/16/25]seed@VM:~/.../crashes$ █
```

图 2: 开头为 C 且长度为 30 段错误样本

id 为 5 的是另一个 SIGSEGV 段错误，当数组开头为 FAS 且数组长度为 6 时，主动触发段错误，代码如下。通过 xxd 分析该样本，其字节内容为“46 41 53 46 46 34”，对应的 ASCII 码即为“FASFF4”，符合开头为 FAS 且长度为 6 的筛选条件。

```
1 else if(str[0] == 'F' && len == 6)  
2     if(str[1] == 'A')  
3         if(str[2] == 'S')  
4             raise(SIGSEGV);
```

```
[12/16/25]seed@VM:~/.../crashes$ xxd id:000005,sig:11,src:000004,op:arith8,pos:2,val:+13  
00000000: 4641 5346 4634                                FASFF4  
[12/16/25]seed@VM:~/.../crashes$ █
```

图 3: 开头为 FAS 且长度为 6 的段错误样本

id 为 0、1、3 的均为栈溢出漏洞，在代码中定义了一个长度为 80 字节的数组 buff，但是这三个样本的长度分别为 132 字节、90 多字节、120 多字节，远超过 buff 数组的容量，从而触发了 gets(buff) 的栈溢出漏洞。下图为 id0 样本的分析图，样本名称中含有 sig:06，这是一个 SIGABRT 中止信号，当检测到溢出破坏了栈上的数据时，GCC 编译器会主动调用 __stack_chk_fail 从而触发 SIGABRT 中止程序

```
[12/16/25]seed@VM:~/.../crashes$ xxd id:000000,sig:06,src:000000,op:havoc,rep:128
00000000: 2b2b 8000 009c 9c9c 2b47 4625 3c2b 2b2b  ++.....+GF%<+++
00000010: 442b 2b80 0c2b 473d 253e 2b00 3e3e 3e9c  D++...+G=%>+.>>>.
00000020: 9c9c 9c9c 9caf 9c9c 9c9c 9c9c f414 3e2b  .....>+
00000030: 2b2b ff85 2f0e 812b 149c 0020 9c6b 949c  ++./...+... .k..
00000040: 9c9c 0022 0064 9c2b 142b 0000 ffff 2b49  ...".d.+...+I
00000050: 0c2b 473d 253e 2b2b 2b0b 1f2b 0e79 2b14  .+G=%>+++...+.y+.
00000060: fc30 492b 1414 2b4e 2b2b 3b9c 9c9c 9c8b  .0I+...N++;.....
00000070: 9cff 7f9c 9c9c 9c9c 9cca 9c9c 9c9c 9c9c  .....
00000080: 9c9c 9c01                                     ....
[12/16/25]seed@VM:~/.../crashes$
```

图 4: 栈溢出漏洞样本

id 为 4、6 的是格式化字符串漏洞，在代码中使用了 `printf(buff)`，当输入中包含格式化字符串时 `printf` 会尝试从栈上读取数据或写入数据，导致程序崩溃。分析样本发现，输入中存在着大量的 25 即格式化字符 “%”，从而非法读取数据，如图所示。

```
[12/16/25]seed@VM:~/.../crashes$ xxd id:000006,sig:06,src:000004+000003,op:splice,rep:64
00000000: 4664 9dde de10 0080 0000 2525 2525 2525  Fd.....%
00000010: 257d 25de dede dec0 dfbc dede e6dd df0c  %}%.....
00000020: ffd6 f8bc debc 6cdf 0080 e9c5 d625 2525  ....l.....%
00000030: 2525 2525 2525 2525 2525 2525 2525 2525  %%%%%%%%%%
00000040: 2525 dede fcde ff00 bcde dee6 2525 2525  %%......%
00000050: 2525 2525 2525 2501 ff40 ffd5 dede      %%%%%%%%%..@....
```

图 5: 格式化字符串漏洞样本

3 Task 2: 基于 AFL++ 测试 Xpdf

3.1 漏洞复现

接下来采用 AFL++ 测试 Xpdf，使用自源码编译的方式安装最新版本的 AFL++，然后下载 Xpdf 源码，并准备若干 PDF 示例文件用于测试。由于 seed Ubuntu 会出现难以解决的环境错误，后续实验在 Ubuntu 22.04 上进行。

首先使用 `afl-clang-fast` 和 `afl-clang-fast++` 作为编译器，对 Xpdf 进行重新配置与构建

```
1 CC=$HOME/AFLplusplus/afl-clang-fast CXX=$HOME/AFLplusplus/afl-clang-
  fast++ ./configure--prefix="$HOME/fuzzing_xpdf/install/"
2 make
3 make install
```

在编译输出中可以看到 AFL++ 的插桩提示，表明目标程序已被插入覆盖率收集代码。随后可使用如下命令启动模糊测试。

```
1 HOME/AFLplusplus/afl-fuzz-i $HOME/fuzzing_xpdf/pdf_examples/-o $HOME/
  fuzzing_xpdf/out/ -s 123 -- $HOME/fuzzing_xpdf/install/bin/pdftotext
  @@ $HOME/fuzzing_xpdf/output
```

模糊测试结果如图所示，获得了一个 crash。

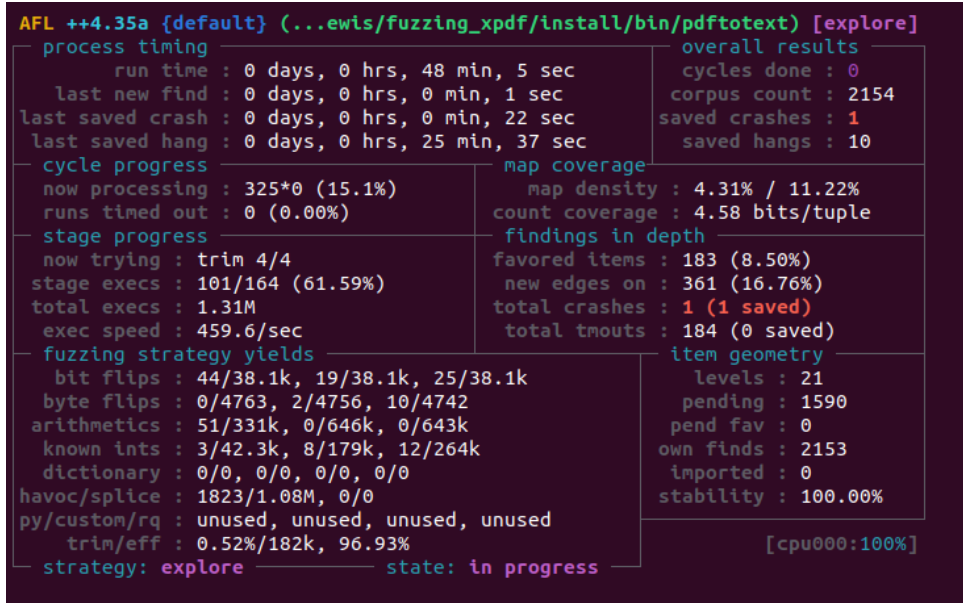


图 6: 模糊测试 Xpdf 结果

在获取了样本之后，复现 crash 并使用 gdb 分析导致程序崩溃的具体原因。首先，需要重新编译 Xpdf，引入调试信息以获得完整的符号和栈回溯信息：

```

1 rm -r $HOME/fuzzing_xpdf/install
2 cd $HOME/fuzzing_xpdf/xpdf-3.02/
3 make clean
4 CFLAGS="-g -O0" CXXFLAGS="-g -O0" ./configure --prefix="$HOME/
  fuzzing_xpdf/install/"
5 make
6 make install
  
```

然后，使用 gdb 对触发崩溃的输入样本进行调试，在 gdb 中依次执行 run 和 bt。使用 run 启动程序，崩溃发生后显示段错误 (SegmentationFault) 导致的异常退出，通过 bt 查看栈回溯信息：

```

Error (397): Dictionary key must be a name object
Error (405): Illegal character '>'
Error (408): Dictionary key must be a name object
Error (411): Dictionary key must be a name object
Error (191): Dictionary key must be a name object
Program received signal SIGSEGV, Segmentation fault.
Download failed: Invalid argument. Continuing without source file ./stdio-common/.std
0x00007ffff7f809a7 in __printf_buffer (buf=buf@entry=0x7ffff7ff4a0,
  format=format@entry=0x555555684034 "Error (%d): ", ap=ap@entry=0x7ffff7ff5a0,
  mode_flags=mode_flags@entry=0) at ./stdio-common/vfprintf-internal.c:600
warning: 600 ./stdio-common/vfprintf-internal.c: No such file or directory
(gdb) bt
#0 0x00007ffff7f809a7 in __printf_buffer (buf=buf@entry=0x7ffff7ff4a0,
  format=format@entry=0x555555684034 "Error (%d): ", ap=ap@entry=0x7ffff7ff5a0,
  mode_flags=mode_flags@entry=0) at ./stdio-common/vfprintf-internal.c:600
#1 0x00007ffff7f8073b in _vfprintf_internal (s=0x7ffff7a044e0 <IO_2_1_stderr_>,
  format=0x555555684034 "Error (%d): ", ap=ap@entry=0x7ffff7ff5a0,
  mode_flags=mode_flags@entry=0) at ./stdio-common/vfprintf-internal.c:1544
#2 0x00007ffff7f85f5c in __fprintf (stream=<optimized out>, format=<optimized out>)
  at ./stdio-common/printf.c:32
#3 0x000055555553619e in error (pos=265, msg=0x555555680e5a "Illegal character '>'")
  at Error.cc:29
#4 0x000055555553619e in Lexer::getObj (this=0x5555780e0b0, obj=0x5555780e0b8)
  at Lexer.cc:424
  
```

图 7: 使用 run 后显示段错误

```

Type <RET> for more, q to quit, c to continue without paging.-bt
#14 0x0000555555561da3 in Parser::getObj (this=this@entry=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0,
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:194
#15 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#16 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#17 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#18 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#19 Parser::makeStream (this=this@entry=0x5555780d5c0, dict=dict@entry=0x7ffff7ffca0, fileKey=fileK
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:156
#20 0x0000555555561da3 in Parser::getObj (this=this@entry=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0,
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:194
#21 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#22 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#23 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#24 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
#25 Parser::makeStream (this=this@entry=0x5555780d5c0, dict=dict@entry=0x7ffff7ffca0, fileKey=fileK
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:156
#26 0x0000555555561da3 in Parser::getObj (this=this@entry=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0,
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:194
#27 0x0000555555561da3 in Parser::getObj (this=0x5555780d5c0, obj=obj@entry=0x7ffff7ffca0, fileKey=f
  keyLength=keyLength@entry=0, objNum=objNum@entry=2, objGen=0) at Parser.cc:64
  
```

图 8: 使用 bt 查看栈信息

根据 bt 输出，这是一个无限递归导致的栈溢出，观察栈帧 #14 到 #39，会发现一个重复循环模式，且给 Parser::getObj 函数的参数 objNum 在多次调用中均为 2，这说明出现了循环调用。程序在不断地循环调用以下几个函数：

- Parser::getObj: 尝试解析一个对象
- XRef::fetch: 尝试通过交叉引用表获取对象
- Object::dictLookup: 在字典对象中查找某个键
- Parser::makeStream: 尝试构建流对象

循环的流程如下:

1. Parser::getObj 调用 Parser::makeStream
2. Parser::makeStream 内部需要获取对象, 接着调用 Object::dictLookup 和 XRef::fetch
3. XRef::fetch 再次调用 Parser::getObj, 这会导致无限递归

接着为了动态观察递归过程, 我们需要在导致死循环的函数入口处设置断点, 即在 Parser::getObj 处下断点。设置好断点后重新启动程序, 程序在 Parser::getObj 处停止, 单点调试程序, 发现程序每次都在同一个断点 Parser::getObj 处停下, 说明了函数确实发生了循环调用。

```
1 b Parser::getObj
2 run
3 c
```

```
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, Parser::getObj (this=0x55555591c010, obj=0x7ffffffdb38, fileKey=0x0,
    encAlgorithm=cryptRC4, keyLength=0, objNum=0, objGen=0) at Parser.cc:41
41      int objNum, int objGen) {
(gdb) c
Continuing.
Error: PDF file is damaged - attempting to reconstruct xref table...

Breakpoint 1, Parser::getObj (this=0x55555591c010, obj=0x7ffffffda20, fileKey=0x0,
    encAlgorithm=cryptRC4, keyLength=0, objNum=0, objGen=0) at Parser.cc:41
41      int objNum, int objGen) {
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=0x55555591c010, obj=obj@entry=0x7ffffffd990,
    fileKey=fileKey@entry=0x0, encAlgorithm=encAlgorithm@entry=cryptRC4,
    keyLength=keyLength@entry=0, objNum=objNum@entry=0, objGen=0) at Parser.cc:41
41      int objNum, int objGen) {
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=0x55555591c010, obj=obj@entry=0x7ffffffd990,
    fileKey=fileKey@entry=0x0, encAlgorithm=encAlgorithm@entry=cryptRC4,
    keyLength=keyLength@entry=0, objNum=objNum@entry=0, objGen=0) at Parser.cc:41
41      int objNum, int objGen) {
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=0x55555591c010, obj=0x7ffffffdad8, fileKey=0x0,
    encAlgorithm=cryptRC4, keyLength=0, objNum=0, objGen=0) at Parser.cc:41
41      int objNum, int objGen) {
```

图 9: 单点调试结果

3.2 漏洞分析

1. 漏洞类型：不受控制的无限递归导致的栈溢出
2. 现象描述：通过 GDB 调试发现，当程序处理特定的 PDF 样本时，进程因段错误而终止。查看函数调用栈可以观察到，Parser::getObj、Parser::makeStream 和 XRef::fetch 等函数之间形成了递归调用循环。在递归过程中，传递给函数的参数 objNum 始终保持为同一值，且栈帧数量迅速增加直至耗尽栈空间。
3. 漏洞产生根本原因：该漏洞 (CVE-2019-13288) 产生的根本原因在于 Xpdf 的解析器 (Parser) 在处理 PDF 对象的交叉引用结构时，缺乏对递归深度的限制和循环引用的检测机制。

```
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=this@entry=0x55555591c010,
    keyLength=keyLength@entry=0, objNum=objNum@entry=2, objC
41                                int objNum, int objGen) {
(gdb) p objNum
$13 = 2
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=this@entry=0x55555591c010,
    keyLength=keyLength@entry=0, objNum=objNum@entry=2, objC
41                                int objNum, int objGen) {
(gdb) p objNum
$14 = 2
(gdb) c
Continuing.

Breakpoint 1, Parser::getObj (this=this@entry=0x55555591c010,
    keyLength=keyLength@entry=0, objNum=objNum@entry=2, objC
41                                int objNum, int objGen) {
(gdb) p objNum
$15 = 2
(gdb) c
```

图 10: 传递给函数的参数 objNum 始终相同

4 Task 3: 使用 QEMU 模式执行模糊测试

在之前的任务中，我们主要针对具有源代码的程序开展模糊测试。而在真实应用场景中，测试目标往往仅以二进制形式提供，无法获得源码。在任务三中使用 AFL++ 提供的 QEMU 模式，使得在缺乏源码的情况下仍然可以进行覆盖引导的模糊测试。首先在 AFLplusplus 源码目录中为 QEMU 模式安装所需依赖并构建支持模块，接着使用 QEMU 模式执行模糊测试。

使用任务给定的已编译并安装好的 exif 程序作为测试目标，输入样本来自 exif-samples 文件夹中的 JPEG 文件，输入指令启动模糊测试。


```
1 $HOME/AFLplusplus/afl-fuzz -Q -i $HOME/qemu_fuzz/exif-samples/jpg/-o
  $HOME/qemu_fuzz/out -s 123 -- $HOME/qemu_fuzz/install/bin/exif @@
```

```
AFL ++4.35a [default] (/home/lewis/qemu_fuzz/install/bin/exif) [explore]
process timing
  run time : 0 days, 0 hrs, 59 min, 20 sec
  last new find : 0 days, 0 hrs, 1 min, 42 sec
  last saved crash : 0 days, 0 hrs, 15 min, 14 sec
  last saved hang : none seen yet
cycle progress
  now processing : 187*1 (23.6%)
  runs timed out : 1 (0.13%)
stage progress
  now trying : trim 4/4
  stage execs : 107/817 (13.10%)
  total execs : 752k
  exec speed : 251.9/sec
fuzzing strategy yields
  bit flips : 75/741k, 31/741k, 8/741k
  byte flips : 2/92.7k, 1/92.6k, 3/92.6k
  arithmetics : 67/6.48M, 0/13.0M, 2/13.0M
  known ints : 2/833k, 2/3.52M, 5/5.19M
  dictionary : 0/0, 0/0, 0/0, 0/0
  havoc/splice : 377/259k, 0/0
  py/custom/rq : unused, unused, unused, unused
  trim/eff : 72.67%/441k, 99.90%
strategy: explore state: in progress ^C

overall results
  cycles done : 1
  corpus count : 792
  saved crashes : 1
  saved hangs : 0
map coverage
  map density : 2.44% / 5.56%
  count coverage : 2.87 bits/tuple
findings in depth
  favored items : 148 (18.69%)
  new edges on : 210 (26.52%)
  total crashes : 1 (1 saved)
  total tmouts : 16 (0 saved)
item geometry
  levels : 5
  pending : 414
  pend fav : 0
  own finds : 696
  imported : 0
  stability : 100.00%
[cpu000: 50%]

+++ Testing aborted by user +++
[*] Writing /home/lewis/qemu_fuzz/out/default/fastresume.bin ...
[+] fastresume.bin successfully written with 1592352 bytes.
[+] We're done here. Have a nice day!
```

图 11: TASK3 模糊测试结果

AFL++ 在 QEMU 模式下发现 crash 样本后，可以使用 GDB 对相应输入进行调试，分析程序异常的原因：

```
1 gdb--args $HOME/qemu_fuzz/install/bin/exif ./crash
```

gdb 中使用 run 后发现如图所示的错误信息。程序抛出了“realloc(): invalid next size”，它表示程序的堆内存管理器（malloc/realloc 机制）检测到了内存损坏。具体来说，程序在之前的操作中发生了越界写入，数据被覆盖到了下一个内存块的 Size 字段中，于是报出了“下一个内存块大小非法”的信息，并主动调用 abort() 终止了程序，系统发出“Program received signal SIGABRT”的终止信号。

```
(gdb) run
Starting program: /home/lewis/qemu_fuzz/install/bin/exif /home/lewis/qemu_fuzz/out/default/crash
es/id:000000,sig:06,src:000469,time:2646378,execs:567319,op:quick,pos:665,val:+8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
realloc(): invalid next size

Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (no_tid=0, signo=6, threadid=140737352775488) at ./nptl/pthread_ki
ll.c:44
44 ./nptl/pthread_kill.c: No such file or directory.
(gdb) █
```

图 12: gdb 调试结果

使用 bt 指令查看回溯信息，结果如图所示。

```
(gdb) bt
#0 __pthread_kill_implementation (no_tid=0, signo=6, threadid=140737352775488)
    at ./nptl/pthread_kill.c:44
#1 __pthread_kill_internal (signo=6, threadid=140737352775488) at ./nptl/pthread_kill.c:78
#2 __GI___pthread_kill (threadid=140737352775488, signo=signo@entry=6)
    at ./nptl/pthread_kill.c:89
#3 0x00007ffff7c42476 in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26
#4 0x00007ffff7c287f3 in __GI_abort () at ./stdlib/abort.c:79
#5 0x00007ffff7c89677 in __libc_message (action=action@entry=do_abort,
    fmt=fmt@entry=0x7ffff7ddb777 "%s\n") at ../sysdeps/posix/libc_fatal.c:156
#6 0x00007ffff7ca0cfc in malloc_printerr (
    str=str@entry=0x7ffff7dd983e "realloc(): invalid next size") at ./malloc/malloc.c:5664
#7 0x00007ffff7ca4c7c in _int_realloc (av=av@entry=0x7ffff7e1ac80 <main_arena>,
    oldp=oldp@entry=0x5555555b0170, oldsize=oldsize@entry=192, nb=nb@entry=368)
    at ./malloc/malloc.c:4842
#8 0x00007ffff7ca5909 in __GI___libc_realloc (oldmem=0x5555555b0180, bytes=354)
    at ./malloc/malloc.c:3485
#9 0x000055555557427c in exif_mem_realloc_func (d=0x5555555b0180, ds=354) at exif-mem.c:23
#10 0x0000555555574498 in exif_mem_realloc (mem=0x5555555ace70, d=0x5555555b0180, ds=354)
    at exif-mem.c:91
#11 0x000055555556c264 in exif_entry_realloc (e=0x5555555b0120, d_orig=0x5555555b0180, i=354)
    at exif-entry.c:99
#12 0x000055555556c8b4 in exif_entry_fix (e=0x5555555b0120) at exif-entry.c:253
#13 0x0000555555568945 in fix_func (e=0x5555555b0120, UNUSED_PARAM_data=0x0)
    at exif-content.c:246
#14 0x00005555555687dc in exif_content_foreach_entry (content=0x5555555ad4b0,
    func=0x555555568925 <fix_func>, data=0x0) at exif-content.c:215
#15 0x0000555555568a70 in exif_content_fix (c=0x5555555ad4b0) at exif-content.c:289
#16 0x000055555556bf33 in fix_func (c=0x5555555ad4b0, UNUSED_PARAM_data=0x0)
    at exif-data.c:1274
#17 0x000055555556ba63 in exif_data_foreach_content (data=0x5555555ad380,
    func=0x55555556be0d <fix_func>, user_data=0x0) at exif-data.c:1128
#18 0x000055555556bf62 in exif_data_fix (d=0x5555555ad380) at exif-data.c:1281
#19 0x000055555556b49c in exif_data_load_data (data=0x5555555ad380,
    d_orig=0x5555555acef0 "Exif", ds_orig=1156) at exif-data.c:963
#20 0x0000555555573d3e in exif_loader_get_data (loader=0x5555555acea0) at exif-loader.c:401
#21 0x000055555556602c in main (argc=2, argv=0x7fffffffdf88) at main.c:438
```

图 13: 使用 bt 指令查看信息

从栈信息中可以发现，Frame #6 中输出了 realloc(): invalid next size 表示程序的内存管理器 (glibc malloc) 在尝试调整一块内存大小时，检测到下一个内存块的元数据 (Size 字段) 已经被破坏。

```
1 #6 0x00007ffff7ca0cfc in malloc_printerr (
2     str=str@entry=0x7ffff7dd983e "realloc(): invalid next size") at ./
    malloc/malloc.c:5664
```

对于获取的栈信息从下往上追踪漏洞触发的逻辑：

1. Frame #21 main: 程序入口。
2. Frame #20 exif_loader_get_data: 加载 EXIF 数据。
3. Frame #19 exif_data_load_data: 负责将原始二进制数据加载解析为 EXIF 结构。
4. Frame #18 到 Frame #14: 程序正在遍历并试图修复 EXIF 内容。
5. Frame #12 exif_entry_fix: 具体到修复某一个 EXIF 条目。
6. Frame #11 exif_entry_realloc: 修复过程中，发现当前缓冲区不够大，试图请求新的内存。

7. Frame #7 `_int_realloc`: 调用底层的内存分配器进行 `realloc` 操作。此时，分配器检查堆的完整性，发现“下一个块的大小”是无效的（说明已经被之前数据写溢出了），于是触发 Frame #6 的报错并终止程序。

这个漏洞符合了 CVE-2012-2836（基于堆的缓冲区溢出漏洞）的特征。漏洞产生的根本原因在于堆溢出导致的堆元数据损坏。在 Linux 系统中 `glibc` 的堆内存管理器会在每个分配的内存块头部维护元数据，用于记录块的大小和标志位。程序在解析恶意的 EXIF 数据时（`exif_data_load_data` 函数），由于缺乏正确的边界检查，向当前分配的堆缓冲区写入了超出其容量的数据，覆盖了相邻的下一个堆内存块的头部信息，当程序后续试图调整内存大小时，会检查相关内存块的完整性。由于下一个块的 `size` 字段已被之前的溢出操作破坏，为了防止安全利用，主动调用 `abort()` 终止了程序。